DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS****
PROJECT NAME	HEMATOVISION: ADVANCED BLOOD CELL CLASSIFICATION USING TRANSFER LEARNING
MAXIMUM MARKS	4 MARKS

5.1 PROJECT MILESTONES & TASKS

Project Flow:

The user interacts with the UI (User Interface) to choose the image.

The chosen image is analyzed by the model which is integrated with the flask application.

Once the model analyses the input the prediction is showcased on the UI

To accomplish this, we have to complete all the activities listed below,

Data Collection:

Collect or download the dataset that you want to train.

Data pre-processing

||

Data Augmentation

||

Splitting data into train and test

||

Model building

||

Import the model-building libraries

||

Initializing the model

||

Training and testing the model

||

Evaluating the performance of the model

||

Save the model

||

Application Building

||

Create an HTML file

||

Build python code

Collect the dataset:

Download the dataset

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.

This dataset contains 12,500 augmented images of blood cells (JPEG) with accompanying cell type labels (CSV). There are approximately 3,000 images for each of 4 different cell types grouped into 4 different folders (according to cell type). The cell types are Eosinophil, Lymphocyte, Monocyte, and Neutrophil.

Link: https://www.kaggle.com/datasets/paultimothymooney/blood-cells/data

As the dataset is downloaded. Let us read and understand the data properly with the help of some visualization techniques and some analyzing techniques.

Note: There are several techniques for understanding the data. But here we have used some of it. In an additional way, you can use multiple techniques.

Activity 1.1: Importing the libraries:

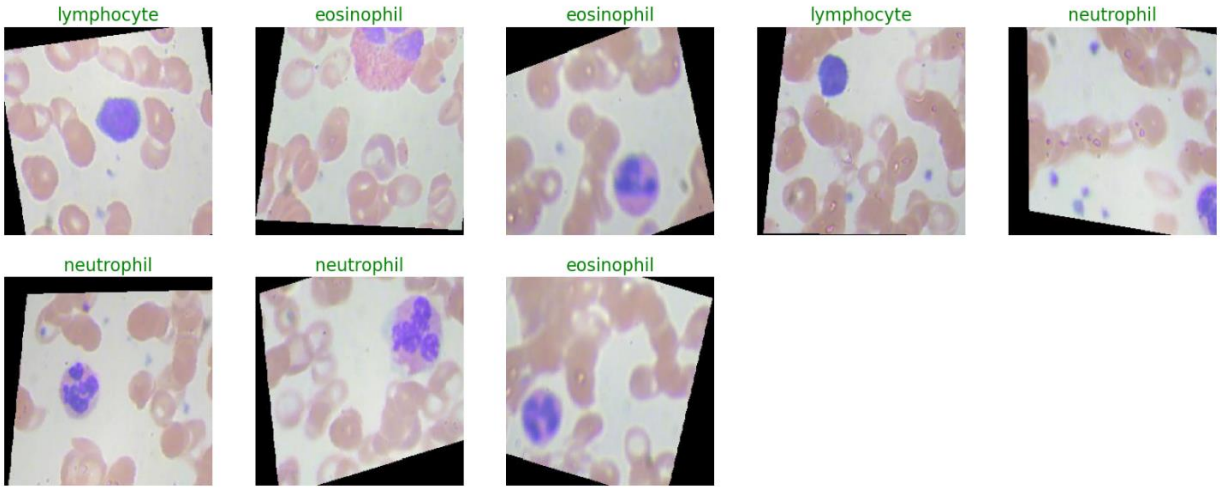
Import the necessary libraries as shown in the image.

```
import os

import pandas as pd

import tensorflow as tf

from tensorflow import keras

import matplotlib.pyplot as plt
```

```
import seaborn as sns

import cv2

from tensorflow.keras.models import load_model

from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

from sklearn.metrics import confusion_matrix, accuracy_score

from sklearn.metrics import classification_report

from sklearn.model_selection import train_test_split

from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

Data Visualization:

The provided Python code imports necessary libraries and modules for image manipulation. It selects a random image file from a specified folder path. Then, it displays the randomly selected image using IPython's Image module. This code is useful for showcasing random images from a directory for various purposes like data exploration or testing image processing algorithms.



SPLIT DATA AND MODEL BUILDING :

The dataset is divided into training and testing sets to ensure proper model evaluation and avoid overfitting. The training set is used to train the model, while the testing set is used to evaluate its performance on unseen data. Data augmentation techniques are applied to improve model generalization and increase dataset diversity.

A pre-trained Convolutional Neural Network (CNN) model is selected for transfer learning. The base model is fine-tuned by adding custom classification layers specific to blood cell

categories. The model is then compiled with an appropriate optimizer and loss function, and trained to achieve high accuracy and reliable performance.

TESTING MODEL & DATA PREDICTION:

After training, the model is evaluated using the testing dataset to measure its performance on unseen blood cell images. The testing phase helps determine the model's generalization ability and reliability in real-world scenarios. Evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix are calculated to assess classification performance.

The trained model predicts the class of new microscopic blood cell images by processing and analyzing input features. It generates output probabilities and assigns the most likely blood cell category with a confidence score. These predictions assist healthcare professionals in making faster and more accurate diagnostic decisions. Overall, testing and prediction ensure that the HematoVision system delivers consistent and dependable results.

APPLICATION BUILDING:

In this section, a web application is developed and integrated with the trained HematoVision model to enable real-time blood cell classification. The application provides a user-friendly interface where users can upload microscopic blood cell images for prediction. The entered image is sent to the saved trained model for processing and classification.

The first task involves building HTML pages that design the front-end of the application. These pages include options to upload images and display prediction results clearly. Proper styling and layout are implemented to ensure ease of use and better user experience.

The second task involves developing the server-side script using a framework such as Flask. The server handles image input, performs preprocessing, loads the trained model, and generates predictions. The predicted blood cell class and confidence score are then displayed on the UI. This integration ensures smooth communication between the front-end interface and the deep learning model.